EDUSPHERE • LMS

EduSphere – Learning Management System

Phase 1: Design Foundation

BACSE202 Database Systems | Project Review 1

TEAM

Name	Register No.
Hamza Ahmad	25BCE2966
Shivansh Agarwal	25BCE2458
V. Aakash	25BCE2460
Sarvesh S	25BCE2106
Samya Basu	25BCE2483

Problem Statement

From scattered data to one source of truth

01

Connected Data

Students, courses, enrollments, quizzes and results are all linked across distributed academic workflows.

02

Unnormalized Anomalies

Duplicates cause severe insert, update, and delete anomalies that corrupt grades and audit trails.

03

Single Source of Truth

A normalized Oracle database in 3NF/BCNF keeps one strictly consistent, ACID-compliant copy.

Scope & Users

Three roles, one platform



Student

- Enroll in courses
- Attempt online quizzes
- View automated & manual results
- Submit assignments & tracking



Instructor

- Manage course content & materials
- Create quizzes & question banks
- Review long-text answers manually
- Define grading schemes & weights



Administrator

- Manage user accounts & RBAC roles
- Manage courses, programs & semesters
- Monitor audit logs & activity traces
- Oversee system-wide grade publishing

Functional Requirements

R1 – R12 at a glance

R1
Sign-up & Login

R2
Role-Based Access

R3
Course Management

R4
Student Enrollment

R5
Course Content

R6
Quiz Creation

R7
Question Bank

R8
Quiz Attempts

R9
Auto-Grading

R10
Manual Review

R11
Results & Grades

R12
Audit & Alerts

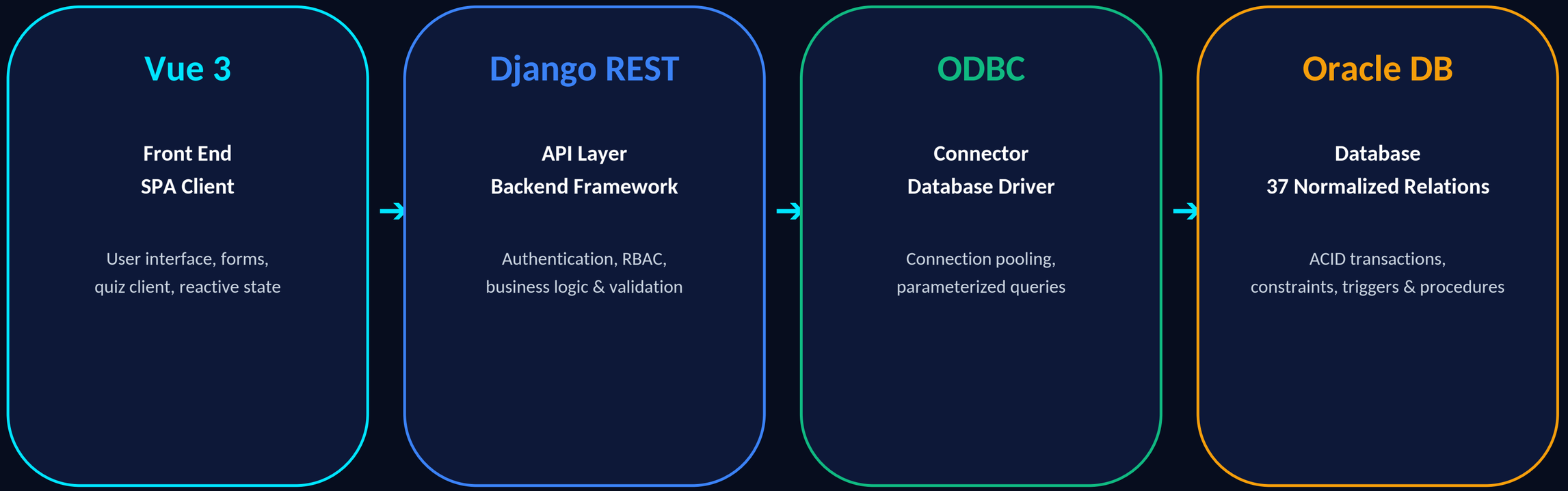
Auto-Grading Rules

8 question types, one grading approach each

#	Question Type	Grading Approach
1	**MCQ_SINGLE**	Auto: selected option must match the exact correct option ID
2	**MCQ_MULTIPLE**	Auto: all correct options selected, penalty/zero if extra options selected
3	**TRUE_FALSE**	Auto: exact binary match with boolean answer key stored
4	**SHORT_TEXT**	Auto: case-insensitive & trimmed match against list of accepted keywords
5	**NUMERIC**	Auto: numerical response within defined tolerance bounds (± delta)
6	**FILL_BLANK**	Auto: multi-blank ordered position match with individual token evaluation
7	**MATCHING**	Auto: bipartite association; each (left, right) pair matched correctly
8	**LONG_TEXT**	Manual review: staged for instructor evaluation with rubrics and inline feedback

System Architecture

Four layers, one safe data path



🔒 **Security Invariant: The browser never connects to the database directly.**

All database operations route through serialized Django endpoints with strict role-based access checks and prepared statements.

ER/EER Overview

Chen notation | 37 relations in 6 domains

8 relations

Identity & Access

7 relations

Academic Structure

5 relations

Course Delivery

7 relations

Assessment Scheme

6 relations

Quiz Core & Grading

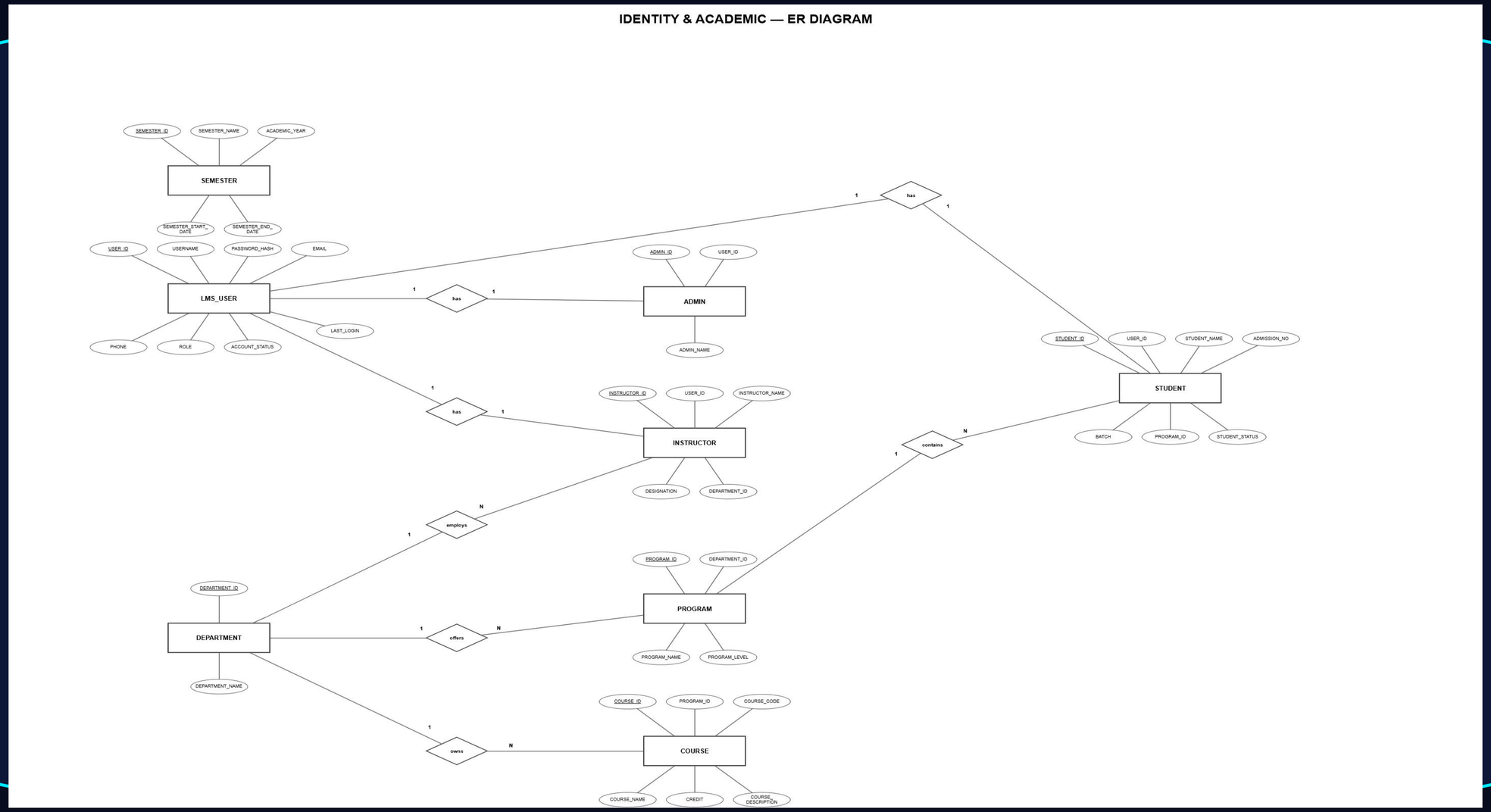
4 relations

Results & Audit

8 + 7 + 5 + 7 + 6 + 4 = 37 RELATIONS (100% IN 3NF / BCNF)

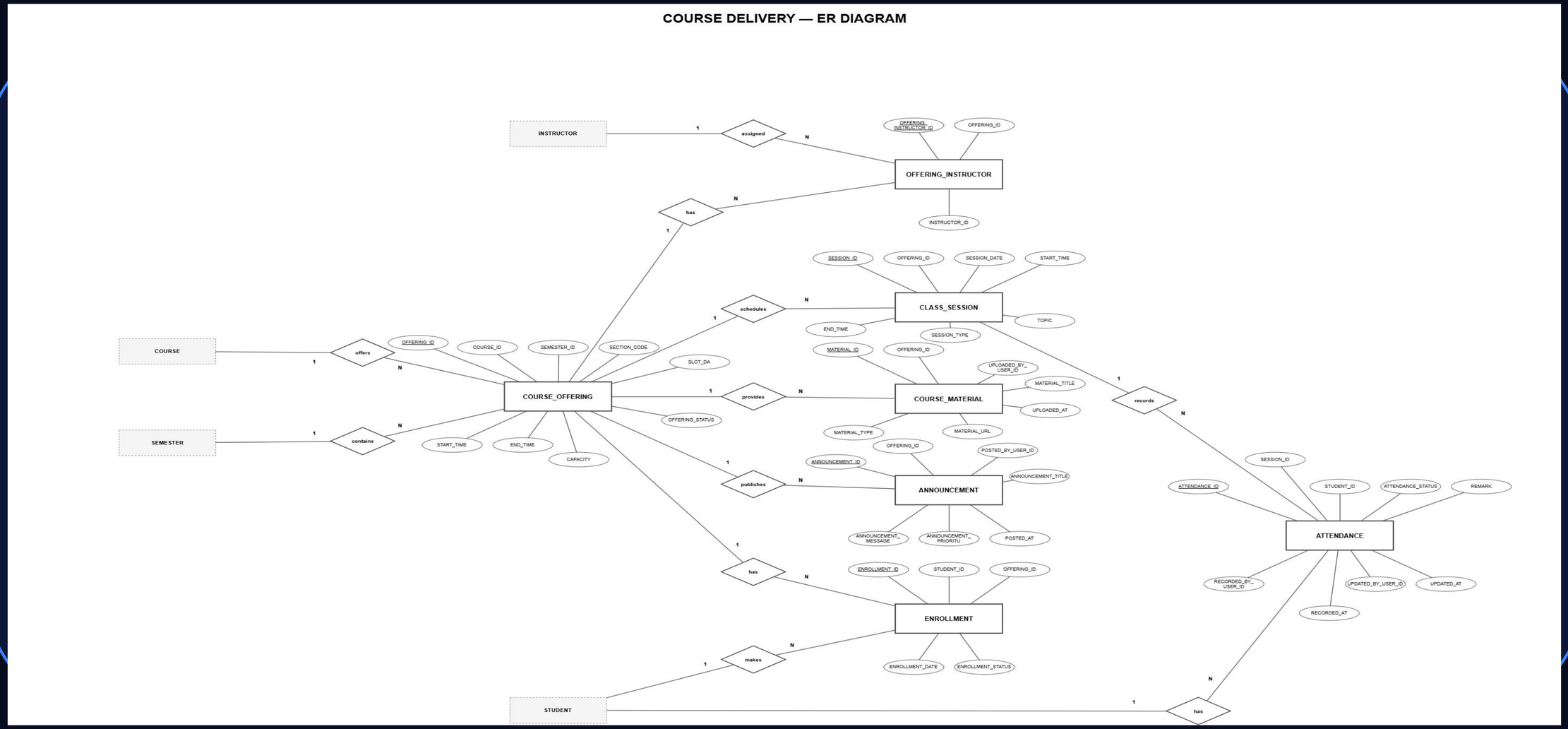
ER Diagram – Identity & Academic Structure

Chen Notation: LMS_USER, Roles, Department, Program, Semester & Course



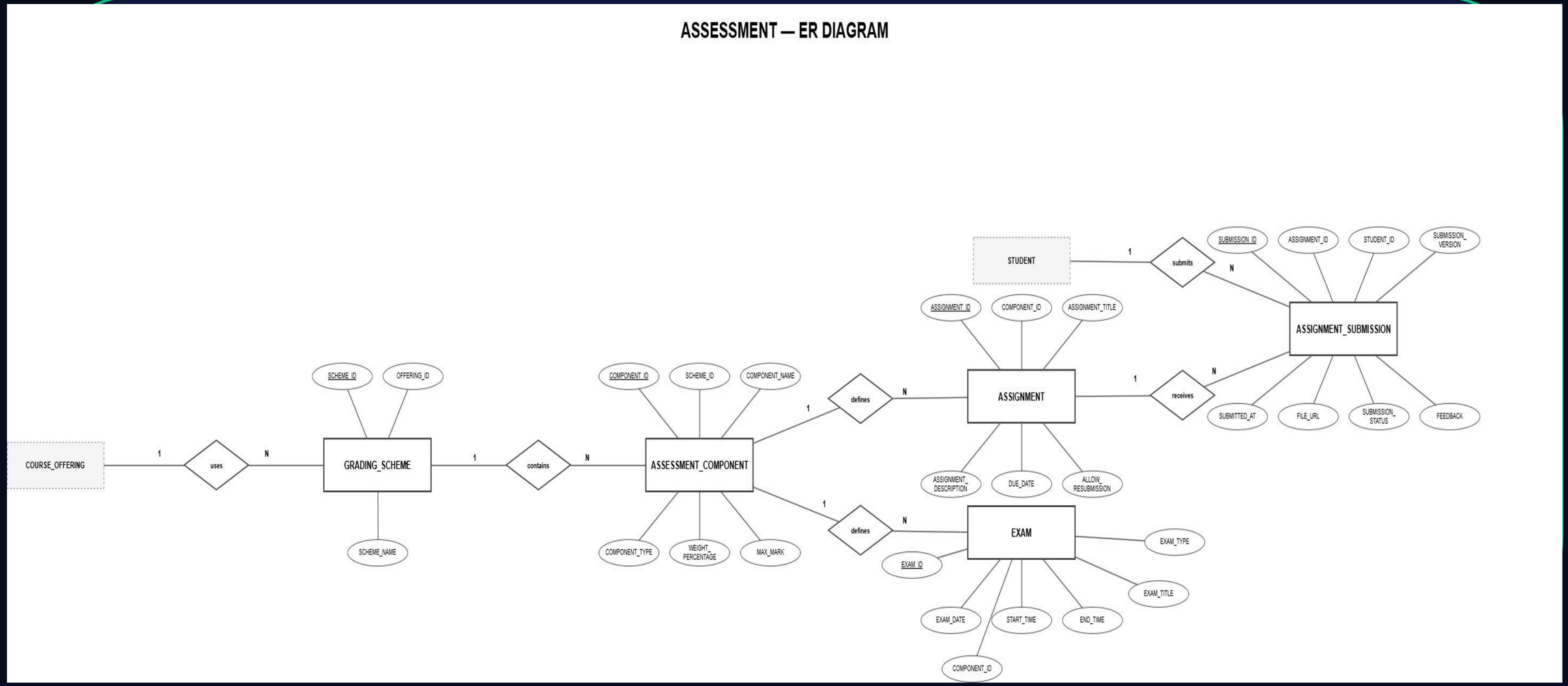
ER Diagram – Course Delivery

Course Offerings, Instructors, Sessions, Attendance, Materials & Announcements



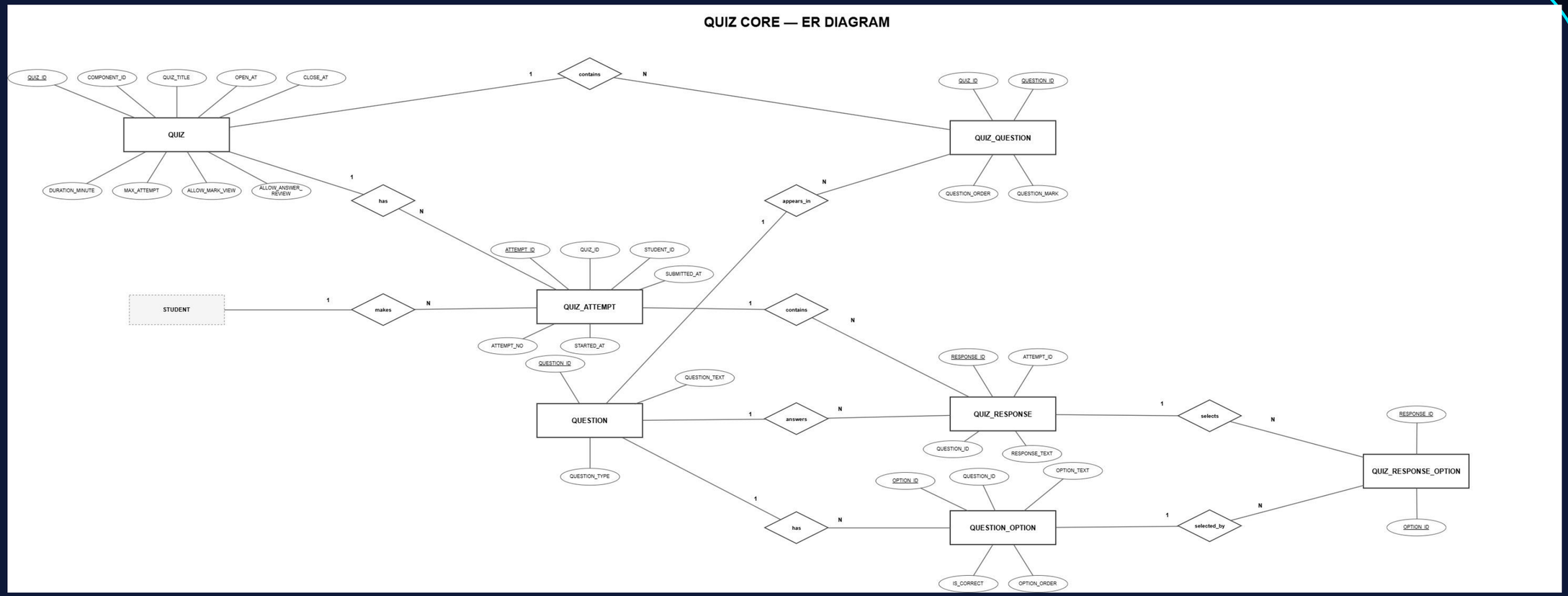
ER Diagram – Assessment & Grading Scheme

Grading Schemes, Assessment Components, Assignments, Submissions & Exams



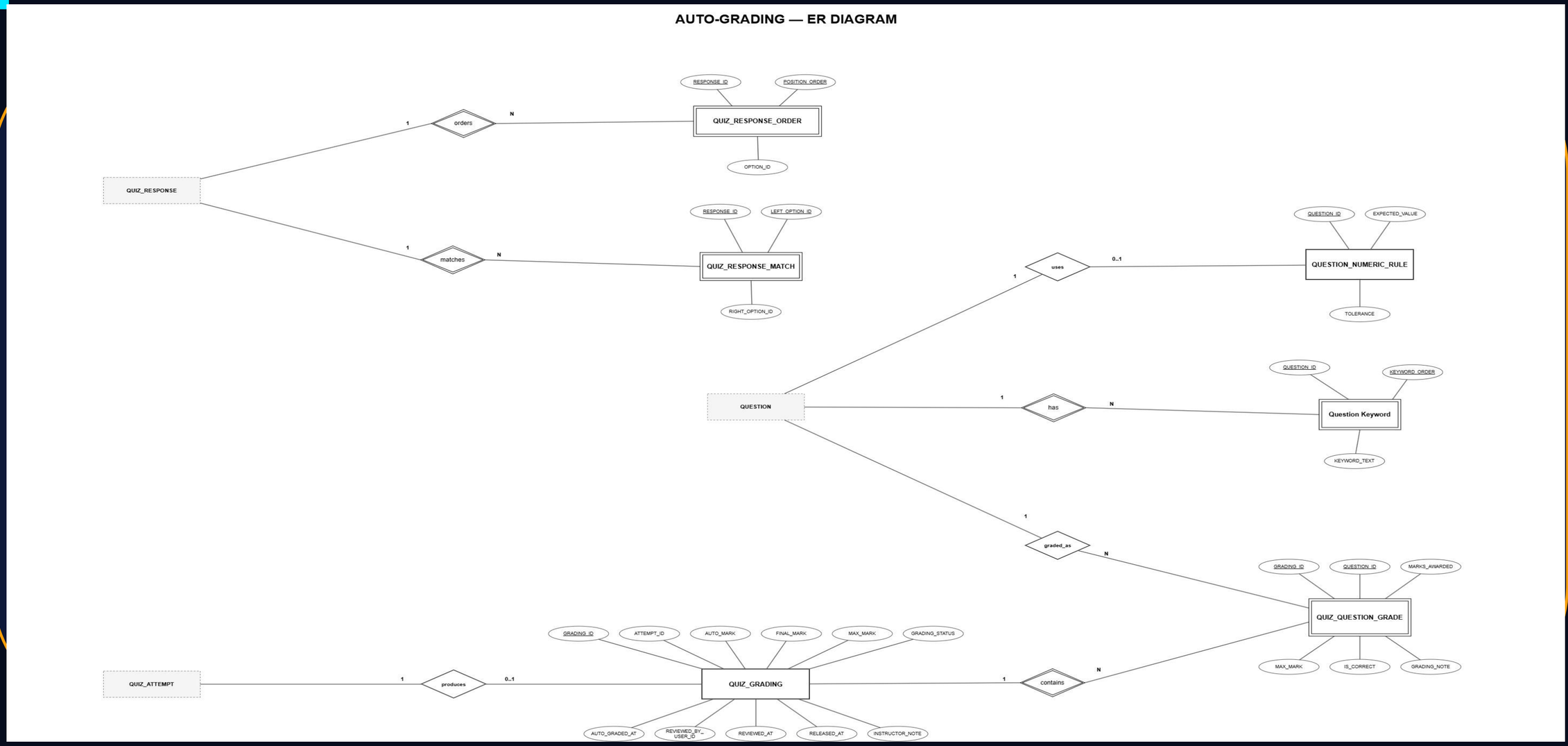
ER Diagram – Quiz Core & Question Banking

Quizzes, Questions, Options, Student Attempts, Responses & Selected Options



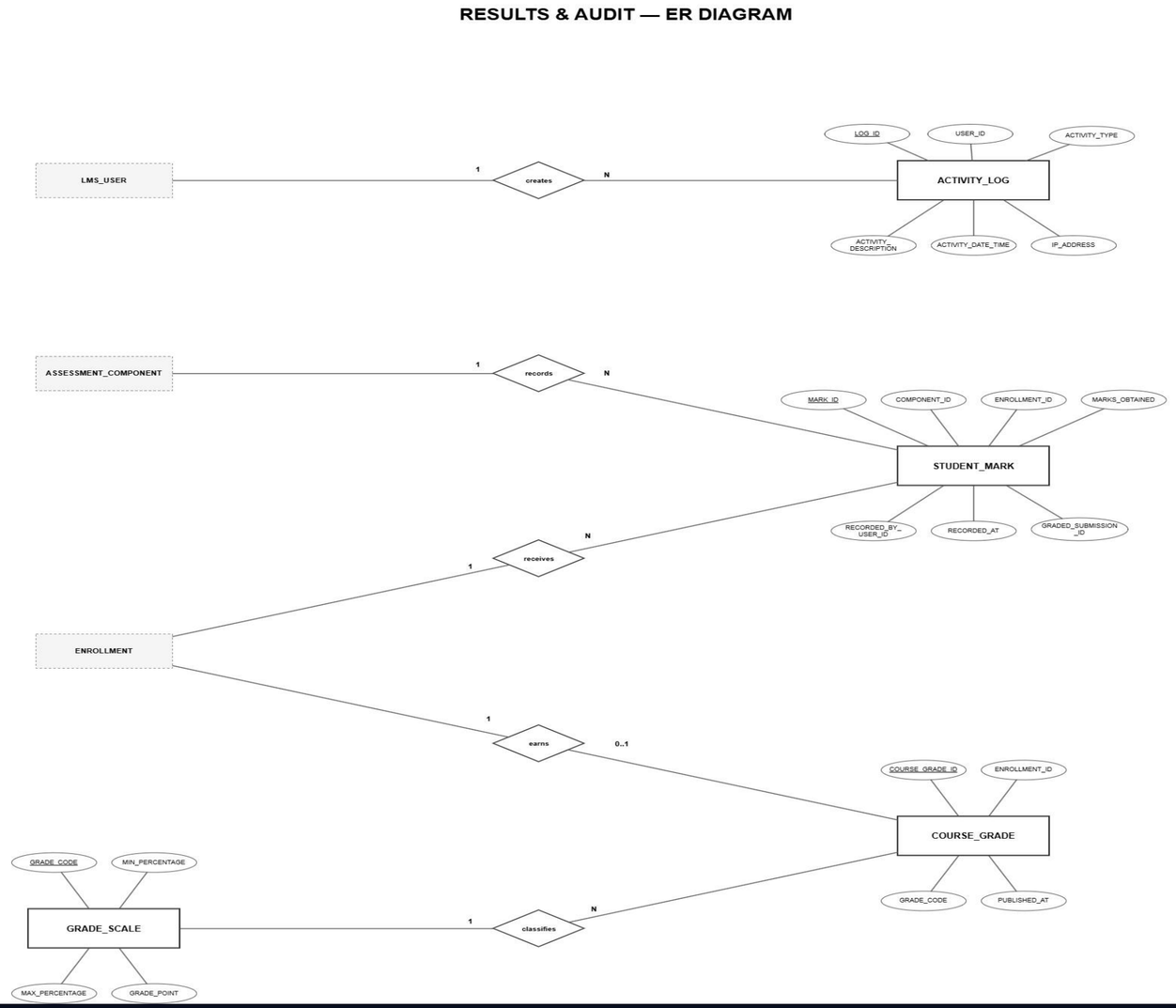
ER Diagram – Auto-Grading & Review Rules

Numeric Tolerance Rules, Keyword Matching, Response Order/Match & Grading Records



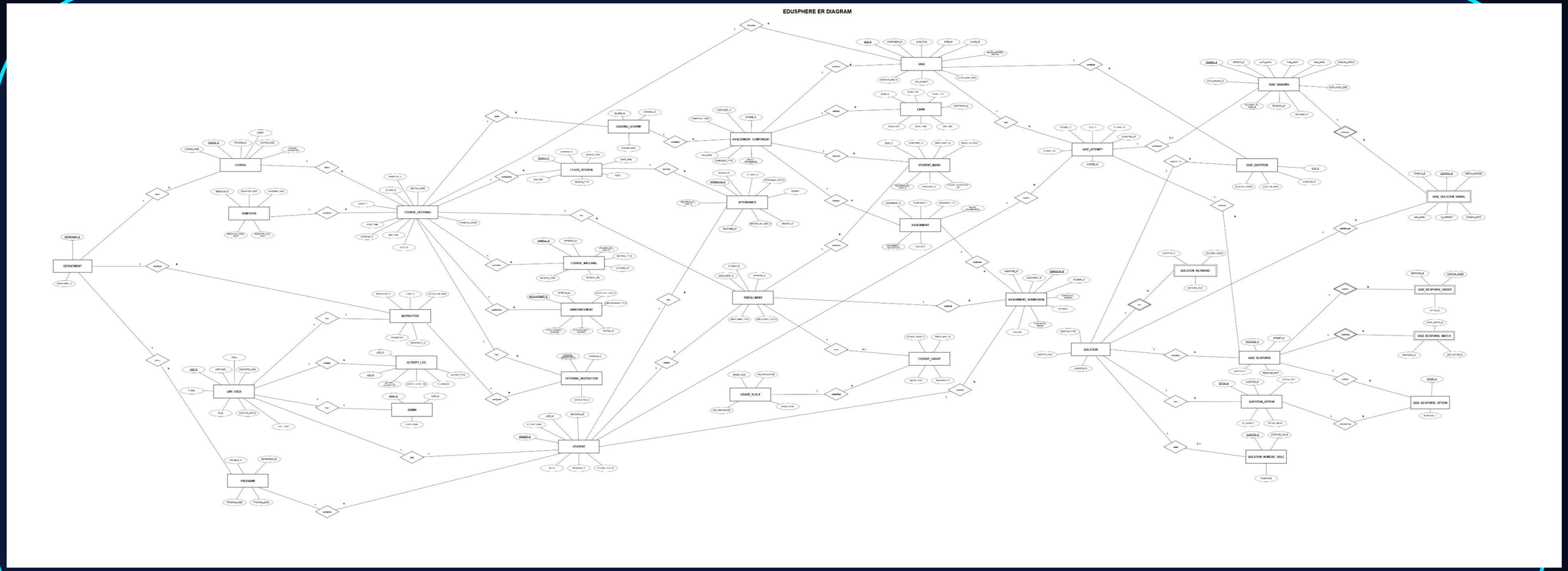
ER Diagram – Results & Audit Logging

Component Marks, Grade Scales, Final Published Course Grades & System Activity Logs



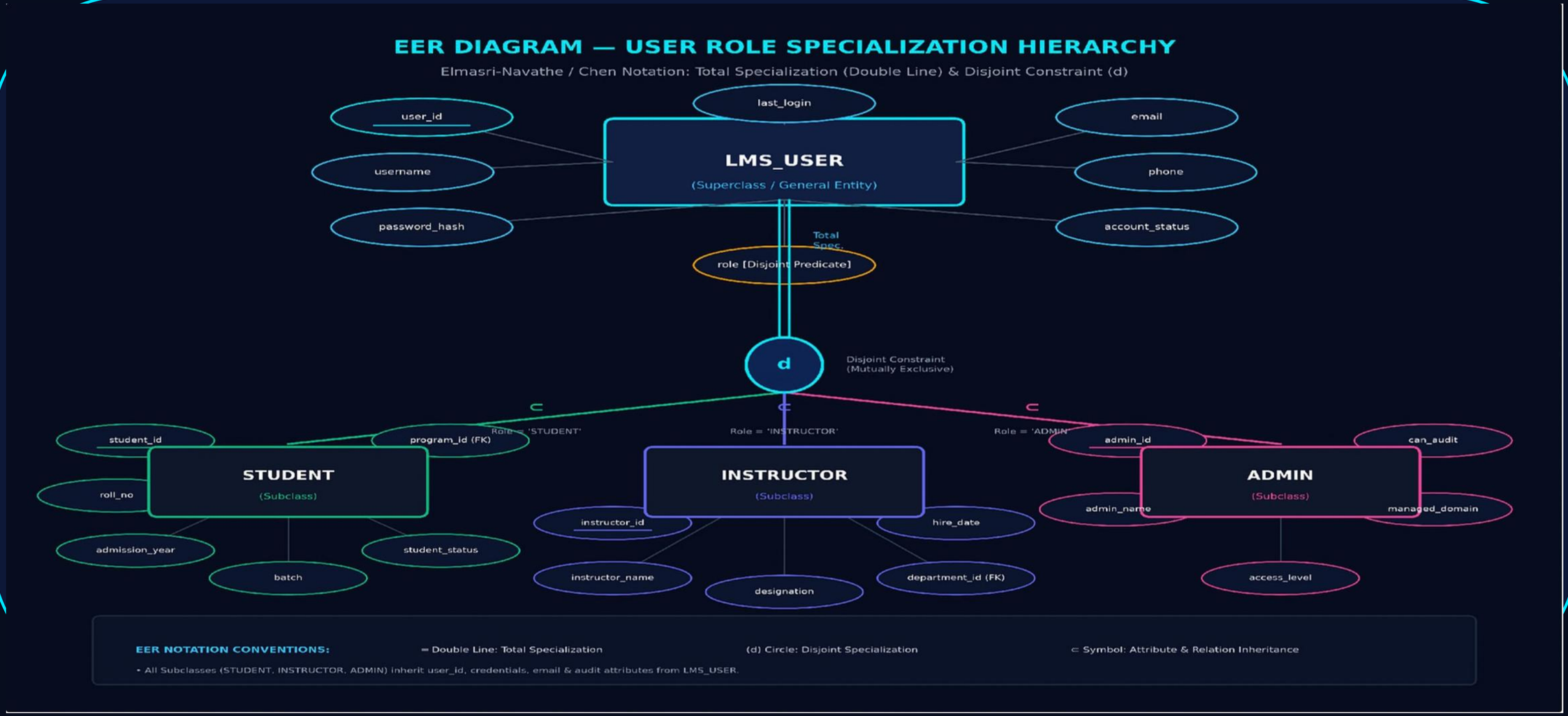
EduSphere Master ER Diagram

Complete Enterprise Schema in Chen Notation (All 37 Relations Combined)



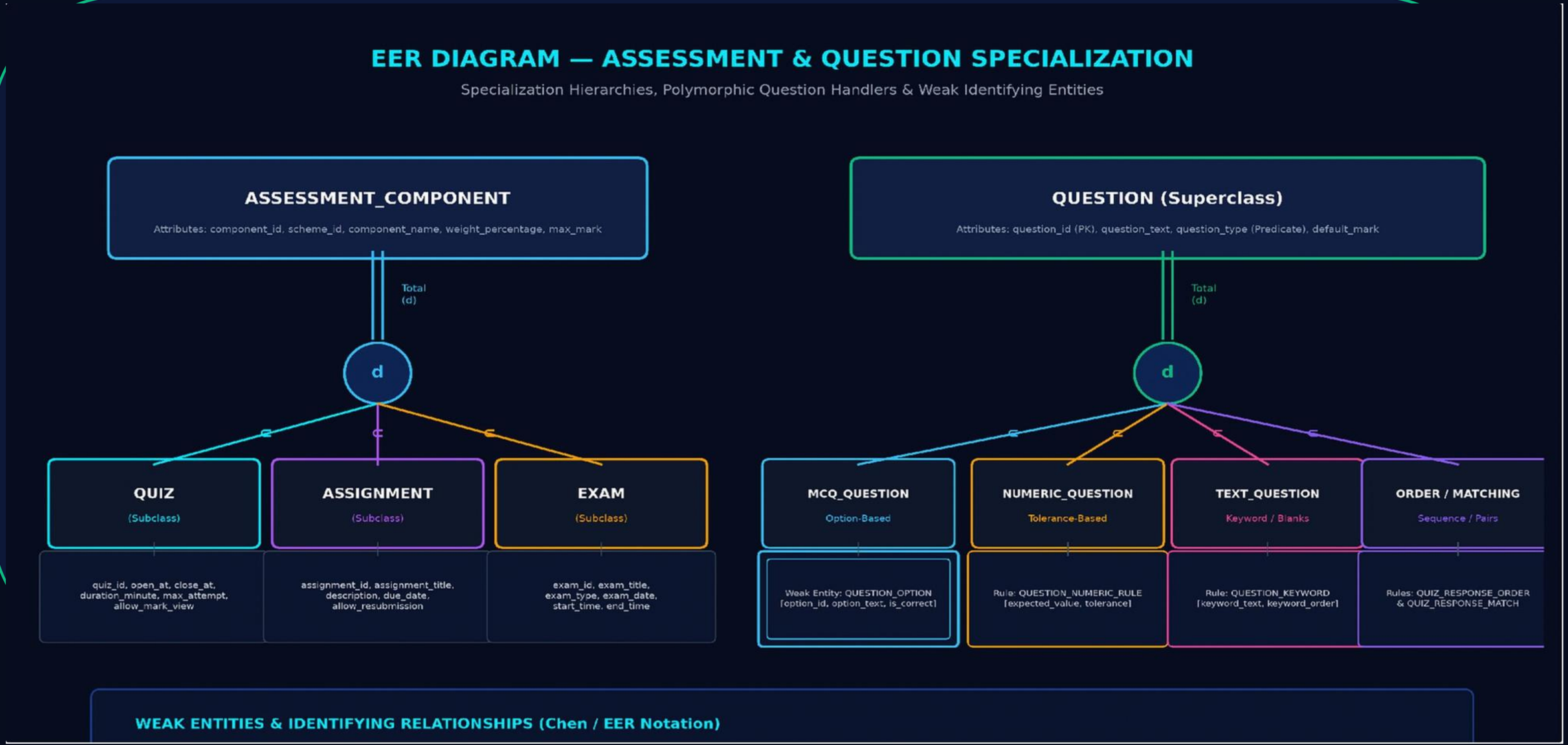
EER Diagram – User Role Specialization Hierarchy

Enhanced ER: Generalization, Total Specialization (Double Line) & Disjointness (d)



EER Diagram – Assessment & Question Polymorphism

Component Specialization, Question Type Handlers & Weak Identifying Entities



ER → Relational Mapping: Rules

8 mapping rules applied step by step to the EduSphere ER/EER model

1 Strong entity → Table

Each strong entity becomes a table; its key attribute becomes the Primary Key (PK).

2 Weak entity → Composite PK

The owner entity's PK is incorporated into the weak entity's composite PK.

3 1:1 → FK + UNIQUE

Place a Foreign Key (FK) on either side and declare it UNIQUE so at most one row matches.

4 1:N → FK on N side

The PK of the 1-side is copied as an FK into the N-side table.

5 M:N → Junction table

A new junction table holds both FKs, with a composite key or a surrogate key.

6 Multivalued attribute → Relation

Each multivalued attribute moves to a separate relation keyed by the owner's PK.

7 Specialization (EER) → Subtype tables

LMS_USER → STUDENT / INSTRUCTOR / ADMIN: user_id is PK/FK to LMS_USER; ROLE is the discriminator; disjoint, total.

8 Composite attribute → Atomic columns

A composite attribute is flattened into one atomic column per component.

Worked Mappings (1/3): Academic Structure

Strong entities and 1:N relationships → tables

ER construct	Resulting table	Columns (bold = PK · FK · PK/FK = part of PK and FK)
Strong entity DEPARTMENT	DEPARTMENT	**DEPARTMENT_ID** (PK), DEPARTMENT_NAME
1:N Department – Program	PROGRAM	**PROGRAM_ID** (PK), DEPARTMENT_ID (FK), PROGRAM_NAME, PROGRAM_LEVEL
Strong entity COURSE (1:N Program – Course)	COURSE	**COURSE_ID** (PK), PROGRAM_ID (FK), COURSE_CODE, COURSE_NAME, CREDITS, COURSE_DESCRIPTION
Subtype STUDENT (of LMS_USER)	STUDENT	**STUDENT_ID** (PK), USER_ID (FK), STUDENT_NAME, ADMISSION_NO, BATCH, PROGRAM_ID (FK), STUDENT_STATUS
Strong entity COURSE_OFFERING	COURSE_OFFERING	**OFFERING_ID** (PK), COURSE_ID (FK), SEMESTER_ID (FK), SECTION_CODE, SLOT_DAY, START_TIME, END_TIME, CAPACITY, OFFERING_STATUS

Worked Mappings (2/3): M:N & Multivalued

Junction tables and multivalued responses → separate relations

ER construct	Resulting table	Columns (bold = PK · FK · PK/FK = part of PK and FK)
M:N Student – Offering	ENROLLMENT	**ENROLLMENT_ID** (PK), STUDENT_ID (FK), OFFERING_ID (FK), ENROLLMENT_DATE, ENROLLMENT_STATUS
M:N Offering – Instructor	OFFERING_INSTRUCTOR	**OFFERING_INSTRUCTOR_ID** (PK), OFFERING_ID (FK), INSTRUCTOR_ID (FK)
M:N Quiz – Question	QUIZ_QUESTION	**QUIZ_ID** (PK/FK), **QUESTION_ID** (PK/FK), QUESTION_ORDER, QUESTION_MARKS
Multivalued response: selected options	QUIZ_RESPONSE_OPTION	**RESPONSE_ID** (PK/FK), **OPTION_ID** (PK/FK)
Multivalued response: ordering	QUIZ_RESPONSE_ORDER	**RESPONSE_ID** (PK/FK), **POSITION_ORDER** (PK), OPTION_ID (FK)
Multivalued response: matching	QUIZ_RESPONSE_MATCH	**RESPONSE_ID** (PK/FK), **LEFT_OPTION_ID** (PK/FK), RIGHT_OPTION_ID (FK)

Worked Mappings (3/3): Rules & Grading

V7 auto-grading constructs → relations

ER construct	Resulting table	Columns (bold = PK · FK · PK/FK = part of PK and FK)
Multivalued keyword set	QUESTION_KEYWORD	**QUESTION_ID** (PK/FK), **KEYWORD_ORDER** (PK), KEYWORD_TEXT
Numeric answer rule	QUESTION_NUMERIC_RULE	**QUESTION_ID** (PK/FK), EXPECTED_VALUE, TOLERANCE
Grading event for a response set	QUIZ_GRADING	**GRADING_ID** (PK), ATTEMPT_ID (FK), AUTO_MARKS, FINAL_MARKS, MAX_MARKS, GRADING_STATUS, AUTO_GRADED_AT, REVIEWED_BY_USER_ID (FK), REVIEWED_AT, RELEASED_AT, INSTRUCTOR_NOTES
M:N Grading – Question	QUIZ_QUESTION_GRADE	**GRADING_ID** (PK/FK), **QUESTION_ID** (PK/FK), MARKS_AWARDED, MAX_MARKS, IS_CORRECT, GRADING_NOTE

Key conventions used in every mapped table

PK is shown in bold · FK marks a foreign key · PK/FK marks a column that is both part of the primary key and a foreign key.
Entity tables use surrogate IDs; pure child / junction tables use composite PKs built from their parent keys.

ER → Relational Mapping: Summary by Domain

37 relations across 6 functional domains

Domain	Relations	Mapped relations by construct type (count)
Identity & Academic	8	**8 entity/subtype:** LMS_USER, DEPARTMENT, PROGRAM, SEMESTER, STUDENT, INSTRUCTOR, ADMIN, COURSE
Course Delivery	7	**4 entity:** COURSE_OFFERING, COURSE_MATERIAL, ANNOUNCEMENT, CLASS_SESSION · **3 associative:** OFFERING_INSTRUCTOR, ENROLLMENT, ATTENDANCE
Assessment	5	**5 entity:** GRADING_SCHEME, ASSESSMENT_COMPONENT, ASSIGNMENT, ASSIGNMENT_SUBMISSION, EXAM
Quiz Core	7	**3 entity:** QUESTION, QUESTION_OPTION, QUIZ · **4 associative/child:** QUIZ_QUESTION, QUIZ_ATTEMPT, QUIZ_RESPONSE, QUIZ_RESPONSE_OPTION
V7 Auto-Grading	6	**2 rule/multivalued:** QUESTION_NUMERIC_RULE, QUESTION_KEYWORD · **2 response child:** QUIZ_RESPONSE_ORDER, QUIZ_RESPONSE_MATCH · **2 grading:** QUIZ_GRADING, QUIZ_QUESTION_GRADE
Results & Audit	4	**4 results/audit:** STUDENT_MARK, GRADE_SCALE, COURSE_GRADE, ACTIVITY_LOG
Total	**37**	**All relations mapped from the ER/EER model**

Conventions

PK shown in bold · FK = foreign key · PK/FK = column that is part of the PK and an FK. Every ER/EER construct maps to exactly one of the 37 relations.

Normalization Strategy & Progression

UNF → 1NF → 2NF → 3NF → BCNF → 4NF → 5NF

Stage	Relations	Outcome
UNF	1 conceptual	One master record, not relational: nested repeating groups
1NF	12	Atomic attributes; repeating groups split into 12 first-stage relations
2NF	24	Partial dependencies removed; 24 relations after 2NF decomposition
3NF	37	Transitive dependencies removed; final 37-relation design
BCNF	37	Reviewed: pass
4NF	37	Reviewed: pass
5NF	37	Reviewed: no further split

Note

Normalization is a decomposition process: relations increase as dependencies are removed. The 3NF result is the exact final 37-relation schema used by the project.

Unnormalized Form (UNF): Master Record

Conceptual master record with nested repeating groups

OFFERING

Instructor [1..n], Student [1..n], Material [1..n], Announcement [1..n], Session [1..n]

ATTENDANCE

Student per Session

ASSESSMENT

Component [1..n]

QUIZ

Question [1..n]; Option [1..n] per question

ATTEMPTS

Attempt [1..n]; Response [1..n]

V7 GRADING

Numeric rule, keyword set, ordering, matching, grading and question-grade groups

Why this is UNF

- One conceptual master record for the whole system
- Repeating groups [1..n] nested inside each other
- Multi-valued attributes such as phone numbers
- Not relational: it is the starting point for decomposition

Next slide

Insertion, update and deletion anomalies caused by this structure.

Unnormalized Form (UNF) & Anomaly Analysis

Flat nested table structure prior to normalization & Classical Anomalies

UNF RELATION: Student_Academic_Records_Flat

(student_id, student_name, email, phone*, program_name, dept_name, {course_code, course_name, offering_id, instructor_name, {quiz_id, quiz_title, attempt_no, score, question_id, response, is_correct}}*)

❌ Violations: Multi-valued phone numbers (*), repeating groups of course offerings ({...}*), and nested quiz attempt tuples.

Insertion Anomaly

- Cannot register a newly approved Course Offering without enrolling at least one student.
- Cannot hire an Instructor without assigning them to an active offering.
- Primary key requirement forces artificial placeholder student records.

Update Anomaly

- Updating a course title or credit requires updating thousands of student enrollment rows.
- Incomplete updates result in conflicting data (e.g., Course listed with 3 credits in one row and 4 in another).
- Destroys data integrity and academic transcript trust.

Deletion Anomaly

- If the only student enrolled in a specialized elective drops the course, deleting their row erases the Course Offering record.
- Historical instructor assignments and syllabus definitions are irreversibly lost.
- Accidental cascade of critical academic metadata.

First Normal Form (1NF): Steps

Atomic attributes, no repeating groups, PK and FKs

1 Atomic attributes

Make every attribute atomic; separate multi-valued phone data into atomic values.

2 Remove repeating groups

Separate the major repeating groups into independent relations, without yet removing partial or transitive dependencies.

3 Choose a PK

Assign keys to the new relations; composite keys are retained where a fact is identified by more than one parent.

4 Add FKs

Add FKs to preserve the links between the decomposed relations.

Flattened 1NF Relation (Partial Sample)

Student ID*	Offering ID*	Quiz ID*	Score
STU101	OFF_01	QUIZ_1	18 / 20
STU101	OFF_01	QUIZ_2	19 / 20
STU101	OFF_02	QUIZ_1	15 / 20
STU102	OFF_01	QUIZ_1	17 / 20

Remaining vulnerability

⚠ student_name ('Hamza Ahmad') and course_title ('Database Systems') repeat in every attempt row.
They depend only on {student_id} or {offering_id}, not the full composite key → violates 2NF.

1NF Result: Domain Split

Master record decomposed into 12 atomic first-stage relations

Domain after 1NF	Relations
Identity / Academic	3
Course Delivery	2
Assessment	2
Quiz Core	2
V7 Auto-Grading	2
Results & Audit	1
Total	**12**

Result of 1NF

- All attributes are atomic and repeating groups are removed
- The master record is decomposed into 12 first-stage relations
- Some relations still contain partial dependencies by design
- Status: 1NF ✓ → next, remove partial dependencies in 2NF

Second Normal Form (2NF): Procedure

Full functional dependency on the whole key

1 List candidate keys

Identify the candidate keys of each relation.

2 Find composite keys

Only relations with a composite key can have partial dependencies.

3 Test non-key attributes

Does each attribute depend on the whole key or only part of it?

4 Decompose

Move partially dependent attributes with their determinant into a new relation.

Functional dependencies examined

FD1: {student_id, offering_id} → student_name, course_name, enrolled_on (partial attributes depend on one component)
FD2: {offering_id, instructor_id} → instructor_name, offering_role (partial on instructor_id)
FD3: {quiz_id, question_id} → question_text, question_type (partial on question_id)
FD4: {quiz_id, attempt_id, question_id} → response, is_correct (full dependency)
FD5: {response_id, option_id} → selected_option (full dependency)

Resulting 2NF relations

Decompose partial dependencies and retain full-dependent facts.

Student/identity facts → STUDENT + LMS_USER
Course structure facts → COURSE + COURSE_OFFERING + SEMESTER
Membership facts → ENROLLMENT + OFFERING_INSTRUCTOR
Quiz facts → QUIZ + QUESTION + QUESTION_OPTION + QUIZ_ATTEMPT
Assessment facts → GRADING_SCHEME + ASSESSMENT_COMPONENT + ASSIGNMENT + EXAM

Result: 24 relations → ready for the 3NF transitive-dependency check.

2NF Check: Composite-Key Relations

Composite-key relations verified for partial dependencies

Relation	Composite key	Why there is no partial dependency
QUIZ_QUESTION	QUIZ_ID + QUESTION_ID	QUESTION_ORDER and QUESTION_MARKS describe a question's membership in a specific quiz
QUIZ_RESPONSE_OPTION	RESPONSE_ID + OPTION_ID	Only the selected-option fact is stored
QUIZ_RESPONSE_ORDER	RESPONSE_ID + POSITION_ORDER	OPTION_ID describes the option at that position
QUIZ_RESPONSE_MATCH	RESPONSE_ID + LEFT_OPTION_ID	RIGHT_OPTION_ID is the match for the left option
QUIZ_QUESTION_GRADE	GRADING_ID + QUESTION_ID	Marks and correctness are per question per grading

Conclusion

2NF ✓ 24 relations
Every non-key attribute depends on the whole key. The remaining transitive dependencies are intentionally removed in the next stage.

Third Normal Form (3NF): Procedure

Elimination of transitive dependencies (X → Y → Z)

Procedure

1. Find non-key → non-key dependencies (X → Y → Z).
2. Create a relation for each determinant and move the dependent attributes there.
3. Leave an FK in the original relation.
4. Repeat until no transitive dependency remains.

Result

Status: 3NF ✓

13 additional relations are introduced by decomposition; 24 + 13 = 37.

The resulting 37 relations are exactly the final project schema.

Transitive dependencies found

Academic: PROGRAM → DEPARTMENT; COURSE → PROGRAM → DEPARTMENT.

Course delivery: COURSE_OFFERING → COURSE.

Results: STUDENT_MARK → GRADE_SCALE.

Quiz V7: QUESTION → KEYWORD / NUMERIC RULE; response → ORDER / MATCH.

Audit: user and activity facts are separated from academic entities.

Decomposed 3NF tables

13 final decompositions added

✓ DEPARTMENT
✓ PROGRAM
✓ COURSE
✓ QUESTION_KEYWORD
✓ QUESTION_NUMERIC_RULE
✓ QUIZ_QUESTION
✓ QUIZ_RESPONSE_OPTION / ORDER / MATCH
✓ QUIZ_GRADING
✓ QUIZ_QUESTION_GRADE
✓ STUDENT_MARK
✓ GRADE_SCALE
✓ COURSE_GRADE
✓ ACTIVITY_LOG

24 + 13 = 37 relations

3NF Design Decisions

Why the decomposed 37-relation design satisfies 3NF

Design decision	Why it satisfies 3NF
DEPARTMENT → PROGRAM → COURSE	Academic descriptions are stored once at their own level
COURSE_OFFERING separate from COURSE	Course metadata is not repeated for every section or semester
OFFERING_INSTRUCTOR and ENROLLMENT	Separate membership facts for instructors and students
ASSESSMENT_COMPONENT + ASSIGNMENT / QUIZ / EXAM	Common attributes in the component; subtype attributes in subtype tables
QUESTION_OPTION, QUIZ_QUESTION, QUIZ_ATTEMPT, QUIZ_RESPONSE + child response relations	Each fact (option, membership, attempt, response, selection) has its own relation
QUIZ_GRADING + QUIZ_QUESTION_GRADE	Grading event kept apart from per-question marks
GPA (SGPA / CGPA)	Calculated from published grades, not stored

Boyce-Codd Normal Form (BCNF)

Every determinant is a superkey

Formal rule

For every non-trivial FD X → Y, X must be a superkey.

3NF allows X → Y when Y is a prime attribute; BCNF removes that relaxation.

Check performed

Every determinant in all 37 relations was compared with the candidate keys of its relation.

EduSphere examples

- Each surrogate PK determines its own row
- LMS_USER.EMAIL is an alternate candidate key
- SEMESTER code is an alternate candidate key

Result

✓ Every non-trivial determinant is a PK or alternate candidate key.

✓ 37 relations reviewed: pass. No decomposition needed.

Fourth Normal Form (4NF)

No independent multi-valued facts in one relation

Formal rule

Relation is in BCNF and has no non-trivial multivalued dependency X ↠ Y | Z.
Independent multi-valued facts must not share a relation.

Check performed

Relations were checked for two or more independent one-to-many facts stored together.

EduSphere examples

- Offering has many instructors → OFFERING_INSTRUCTOR
- Offering has many students → ENROLLMENT
- Quiz has many questions → QUIZ_QUESTION
- Response with multiple options → QUIZ_RESPONSE_OPTION
- Ordered list → QUIZ_RESPONSE_ORDER
- Matching pairs → QUIZ_RESPONSE_MATCH

Result

✓ Each multi-valued fact lives in its own relation.
✓ 37 relations: pass 4NF.

Fifth Normal Form (5NF)

Lossless join, no further decomposition

Formal rule

Every non-trivial join dependency is implied by the candidate keys.
Decomposed tables must rejoin without spurious tuples (lossless join).

Check performed

Associative relations were checked for join dependencies that would require a three-way or further split.

EduSphere examples

- ENROLLMENT, OFFERING_INSTRUCTOR and QUIZ_QUESTION represent relationship facts directly
- Response relations store binary facts that rejoin exactly

Result

✓ No non-trivial join dependency needs decomposition.
✓ 37 relations: no further split; lossless join confirmed.

Normalization Status Summary

Final status of the EduSphere V7 schema

Stage	Status	Relation count
UNF	Conceptual master record, not relational	1 conceptual
1NF	Pass: atomic attributes, repeating groups removed; 12 relations	12
2NF	Pass: partial dependencies removed; 24 relations	24
3NF	Pass: transitive dependencies removed; final 37-relation design	37
BCNF	Pass: every determinant is a candidate key	37
4NF	Pass: no non-trivial MVD	37
5NF	Pass: no further split, lossless join	37

Conclusion & Next Phase

Phase 1 Foundation Summary & Phase 2 Technical Implementation Roadmap

Key Outcomes (Phase 1 Completed):

- ✓ 37 Relations designed across 6 cohesive functional domains
- ✓ Full ER and EER diagrams finalized in rigorous Chen notation
- ✓ ER → Relational Schema mapping rules strictly applied
- ✓ Normalized through 1NF, 2NF, 3NF with verified BCNF, 4NF, 5NF
- ✓ Four-layer Architecture: Vue 3 → Django REST → ODBC → Oracle

Phase 2 Technical Roadmap:

P1 — **Oracle DDL Implementation —** Generate all CREATE TABLE statements with sequences, datatypes & schemas.

P2 — **Integrity Constraints —** Enforce PK, FK with CASCADE/SET NULL, UNIQUE, and CHECK constraints.

P3 — **Triggers & Auditing —** Automate audit logging, timestamping, auto-grading triggers & status transitions.

P4 — **Stored Procedures & PL/SQL —** Bulk enrollment processing, quiz attempt scoring & GPA calculation procedures.

P5 — **Django REST API Endpoints —** Build secure serialization, JWT authentication & OpenAPI documentation.

Thank you! | Questions & Discussion